Homework 10

Rahaf Alabdullah (rahafaa2)

Question 1: Bias-variance trade-off simulation

Random seed: `np.random.default_rng(43532)`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from rlt import RLT_reg

rng = np.random.default_rng(43532)
n_train = 200
n_test = 200
p = 5
n_reps = 100
n_trees = 500
nodesize_grid = [1, 5, 10, 20, 30] #(with mtry fixed at 3)
mtry_grid = [1, 2, 3, 4, 5]        #(with node size fixed at 10)

cov = np.full((p, p), 0.3)
np.fill_diagonal(cov, 1)
mean = np.zeros(p)

test_X = rng.multivariate_normal(mean, cov, size=n_test)
test_f = test_X[:, 0] + test_X[:, 2] + test_X[:, 4]

array_nodesize = np.empty((len(nodesize_grid), n_reps, n_test)) #(with mtry fixed at 3)
array_mtry = np.empty((len(mtry_grid), n_reps, n_test))         #(with node size fixed at 10)

for rep in range(n_reps):
    # training data
    train_X = rng.multivariate_normal(mean, cov, size=n_train)
    train_y = train_X[:, 0] + train_X[:, 2] + train_X[:, 4] + rng.standard_normal(n_train)

    # loop over nodesize_grid with mtry = 3
    for i, nodesize in enumerate(nodesize_grid):
        forest = RLT_reg(n_estimators=n_trees, mtry=3, min_samples_leaf=nodesize,
                         resample_prob=1, resample_replace=True,
                         random_state=rng.integers(1e6), n_jobs=1)
        forest.fit(train_X, train_y)
        array_nodesize[i, rep, :] = forest.predict(test_X)

    # loop over mtry_grid with min_samples_leaf = 10
    for j, mtry in enumerate(mtry_grid):
        forest = RLT_reg(n_estimators=n_trees, mtry=mtry, min_samples_leaf=10,
                         resample_prob=1, resample_replace=True,
                         random_state=rng.integers(1e6), n_jobs=1)
        forest.fit(train_X, train_y)
        array_mtry[j, rep, :] = forest.predict(test_X)

mean_preds_nodesize = array_nodesize.mean(axis=1)
bias2_nodesize = ((mean_preds_nodesize - test_f) ** 2).mean(axis=1)  #
var_nodesize = array_nodesize.var(axis=1).mean(axis=1)
table1 = pd.DataFrame({
    "nodesize": nodesize_grid,
    "bias2": bias2_nodesize,
    "variance": var_nodesize
})
mean_preds_mtry = array_mtry.mean(axis=1)
bias2_mtry = ((mean_preds_mtry - test_f) ** 2).mean(axis=1)
var_mtry = array_mtry.var(axis=1).mean(axis=1)
table2 = pd.DataFrame({
    "mtry": mtry_grid,
    "bias2": bias2_mtry,
```

```
    "variance": var_mtry
})
print(f"Table 1: Bias^2 and Variance vs Node Size\n{table1}\n")
print(f"Table 2: Bias^2 and Variance vs mtry\n{table2}\n")

fig1, ax1 = plt.subplots()
ax1.plot(nodesize_grid, bias2_nodesize, marker="o", label="Squared Bias")
ax1.plot(nodesize_grid, var_nodesize, marker="o", label="Variance")
ax1.set(xlabel="Node Size", ylabel="Value", title="Bias-Variance Tradeoff vs Node Size")
ax1.legend(); plt.tight_layout()
fig2, ax2 = plt.subplots()
ax2.plot(mtry_grid, bias2_mtry, marker="o", label="Squared Bias")
ax2.plot(mtry_grid, var_mtry, marker="o", label="Variance")
ax2.set(xlabel="mtry", ylabel="Value", title="Bias-Variance Tradeoff vs mtry")
ax2.legend(); plt.tight_layout()
plt.show()
```

(a) Averaged squared bias and variance over the 200 fixed test points (100 repetitions, 500 trees, 200 observations drawn with replacement per tree).

Node size	Bias^2	Variance
1	0.116	0.200
5	0.118	0.195
10	0.127	0.181
20	0.170	0.166
30	0.236	0.160

`mtry`	Bias^2	Variance
1	0.451	0.114
2	0.189	0.146
3	0.128	0.183
4	0.110	0.212
5	0.102	0.234

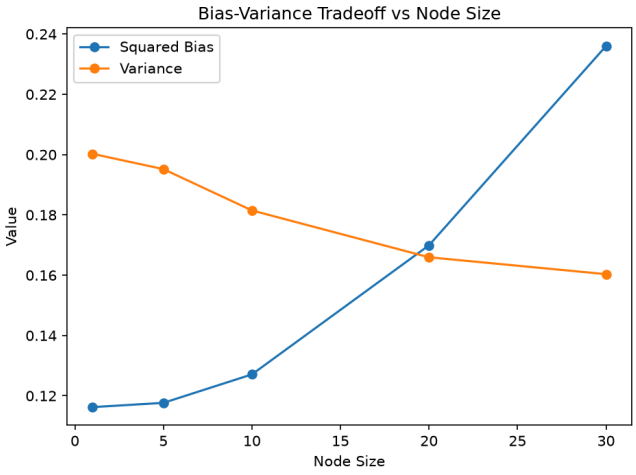


Figure 1: Squared bias and variance vs node size

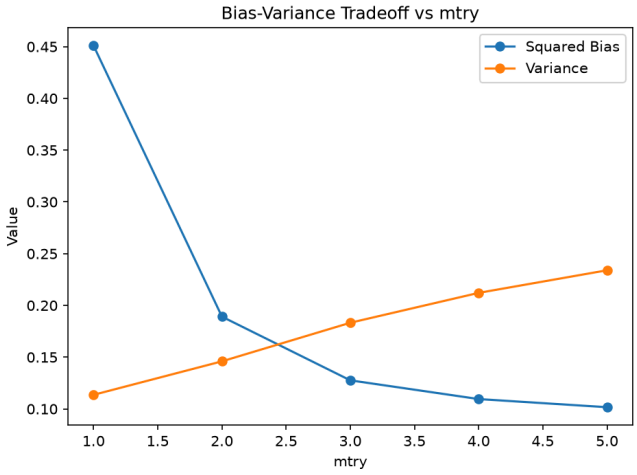


Figure 2: Squared bias and variance vs `mtry`

(b) A large node size (30) means bigger terminal nodes average more observations, giving smoother trees, which leads to low variance but high bias. For large `mtry`, strong variables X_1, X_3, X_5 are chosen more often, so bias is lower, but trees become more alike, so the averaging reduces variance less. If each tree drew 100 observations instead of 200, the best node size would likely move smaller, as smaller nodes would be needed to capture the variability in the data.

Question 2: Tuning random forests for MNIST

Random seed: `np.random.default_rng(43532)`. Training data: 327 observations (digits 1, 6, 7 from the first 1,000); test data: 317 observations. Predictors are the first 20 principal components of the training pixels (centered, not scaled); test pixels are projected with the training means and loadings.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix
from rlt import RLT_cla

data_dir = Path(__file__).parent / "data"
df = pd.read_csv(data_dir / "mnist-first2000.csv.gz")
rng = np.random.default_rng(43532)

train, test = df.iloc[:1000], df.iloc[1000:]
train = train[train["Digit"].isin([1, 6, 7])]
test = test[test["Digit"].isin([1, 6, 7])]
X_train = train.drop(columns="Digit").to_numpy()
y_train = train["Digit"].to_numpy()
X_test = test.drop(columns="Digit").to_numpy()
y_test = test["Digit"].to_numpy()

pca = PCA(n_components=20)
PC_train = pca.fit_transform(X_train)
PC_test = pca.transform(X_test)

forest = RLT_cla(n_estimators=1000, resample_prob=1, resample_replace=True,
                 random_state=rng.integers(1e6), n_jobs=1)
forest.fit(PC_train, y_train)
OBB_misclassification_rate = forest.oob_error_
print(f"OBB Misclassification Rate of the default forest: {OBB_misclassification_rate:.3f}")

nmin_grid = [1, 3, 5, 10, 20]
mtry_grid = [1, 3, 5, 10, 20]
alltune = pd.MultiIndex.from_product([nmin_grid, mtry_grid], names=["nmin", "mtry"])
allerror = np.zeros(len(alltune))
for k, setting in enumerate(alltune.to_frame().itertuples(index=False)):
    forest = RLT_cla(n_estimators=1000, mtry=setting.mtry, min_samples_leaf=setting.nmin,
                     resample_prob=1, resample_replace=True,
                     random_state=rng.integers(1e6), n_jobs=1)
    forest.fit(PC_train, y_train)
    allerror[k] = forest.oob_error_
oob_table = alltune.to_frame(index=False).assign(OOB_error=allerror)
print(oob_table.pivot(index="nmin", columns="mtry", values="OOB_error").round(3))

best_nmin, best_mtry = alltune[np.argmin(allerror)]
forest_best = RLT_cla(n_estimators=1000, mtry=best_mtry, min_samples_leaf=best_nmin,
                 resample_prob=1, resample_replace=True,
                 random_state=rng.integers(1e6), n_jobs=1, importance="permute")
forest_best.fit(PC_train, y_train)
importance_df = pd.DataFrame({
    "Variable": [f"PC{i}" for i in range(1, 21)],
    "VI": forest_best.feature_importances_
}).sort_values("VI", ascending=False)
print(f"OBB Misclassification Rate of the best forest: {forest_best.oob_error_:.3f}")
ax = importance_df.plot.bar(x="Variable", y="VI", legend=False)
plt.title("Variable Importance of PCs")
plt.show()

pred_test = forest_best.predict(PC_test)
test_error = np.mean(pred_test != y_test)
conf_matrix = pd.crosstab(pd.Series(y_test, name="True"),
                          pd.Series(pred_test, name="Predicted"))
print(f"Confusion Matrix:\n{conf_matrix}\n")
print(f"Test Misclassification Rate: {test_error:.3f}")
```

(a) With the default `mtry` and node size, the OOB misclassification rate is **0.040**.

(b) OOB misclassification rate for each setting (rows: node size, columns: `mtry`):

Node size	`mtry` = 1	3	5	10	20
1	0.015	0.037	0.037	0.043	0.049
3	**0.015**	0.037	0.040	0.046	0.049
5	0.018	0.040	0.037	0.043	0.049
10	0.021	0.040	0.040	0.040	0.049
20	0.021	0.037	0.037	0.046	0.052

Selected setting (minimum OOB error): `mtry` = 1, node size = 3. Refitting this setting with permutation importance gives an OOB misclassification rate of 0.024.

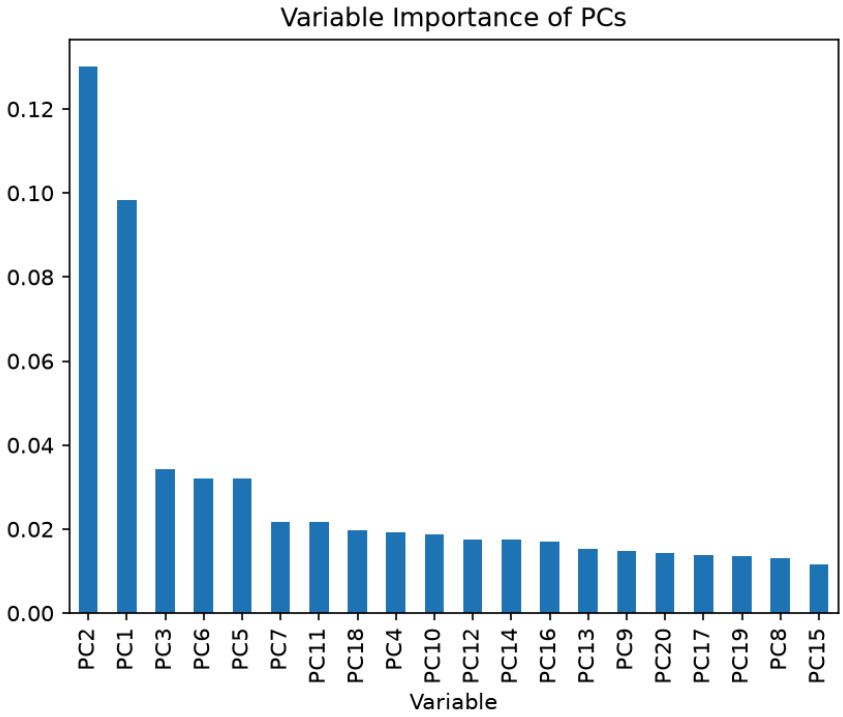


Figure 1: OOB permutation variable importance of the 20 PCs for the selected forest

The most important PCs are PC2, PC1, and PC3. These are the early PCs with the largest variance, which help separate digits 1, 6, and 7. The tuned OOB misclassification rate is lower than part (a), indicating better performance.

(c) Test misclassification rate: **0.013**.

True (rows) / Predicted (columns)	1	6	7
1	102	0	2
6	0	106	0
7	0	2	105

Digit 7 is most confused with digit 6, and digit 1 is most confused with digit 7. The test error is close to the OOB error from part (b), indicating that the model generalizes well to unseen data.

Question 3: Planning a random-forest analysis skill

(a) I would specify in the skill what is the nature of task (eg., classification or regression), the tools, like rlt in Python. Also, I will clearly state how the error is judged; OOB for tuning and then a test set used once at the end. For reproducibility, I will directed it to set a seed and report it. For tuning parameters, I will specify which ones I want it to tune, such as mtry and node size, and specify which ones I would like to be default. For the report, I will clearly state the table and figure style, page limit, code shown or hidden.

(b)

1. Check the data, identify the response and the task type, report n, p and the class balance, and handle missing values and categorical variables before fitting.
2. Use rlt ordinary random forests. Keep every parameter I don't name at its default.
3. Use at least 500 trees (1,000 for the final model). Number of trees is not a tuning parameter, so check that OOB error has levelled off and don't tune it.
4. Use bootstrap samples the same size as the training set.
5. Tune mtry over a small grid from 1 to p.
6. Tune the node size over a small grid.
7. Choose the setting by OOB error using training data only. Touch the test set once, at the end.
8. Report OOB permutation importance, for the selected model only, as a sorted bar chart. Say in the text that importance shows predictive usefulness.
9. Report the test error, and for classification also a confusion matrix. Comment on which classes are confused and how the test error compares with the OOB error.
10. In the report, start with a short summary of the selected settings, the OOB and test error, and the top variables. Use my plot-formatting skill for every figure, show the seed, and keep the report under N pages, with code in an appendix.